MAE 5810 Homework 2 Report

Daniel Whitman
Robot Perception (MAE 5810)
10/02/2026

1 INTRODUCTION

The objective of this assignment is to gain familiarity with the creation, structure learning, and parameter learning of Bayesian networks (BNs) through gathered data. Once properly constructed, these models are remarkably powerful for inferring results from observed data, and can act as a flexible tool in robotic perception applications. The usage that this homework assignment is derived from is in using a BN to predict the propagation loss (PL) or range (R) of an underwater sonar acoustic measurement given different environmental conditions and sensor observations [1].

In acoustic sensor measurements, which are observed due to pressure variations in the medium against the sensor, the following acoustic propagation model based on a three-dimensional, time-dependent wave equation [2] is used:

$$\frac{\partial^2 p}{\partial r^2} + \frac{1}{r}\frac{\partial p}{\partial r} + \frac{\partial^2 p}{\partial z^2} + k^2(z)p = 0 \tag{1}$$

where p is acoustic field pressure, r is the horizontal distance from a point source, z is the depth below the ocean surface, $k = 2\pi f/c = 2\pi/\lambda$ is the wave number, λ is the wavelength, $f = \omega/(2\pi)$, and ω is the source frequency.

From this, the propagation loss (PL) is defined, in decibels (dB), as

$$PL = -20\log|p| \tag{2}$$

and, in the far-field assumption of the acoustic modeling [2] is a function of the following variables:

1. Range (R)
2. Depth (Z)
3. Sea Floor (SF)
4. Bottom Density (BD)
5. Bottom Sound Speed (V)
6. Source Frequency (F)

In relation to the BN, these six variables correspond to the inputs to be observed and the propagation loss corresponds to the final output of the system. In the network model, each variable is represented as a node, and the connections between then are represented as the edges of the graph.

2 DESCRIPTION OF DATA

For this homework, the provided dataset was an simulated from the governing equations in (1) and packaged into a MATLAB `.mat` file. As the data was simulated, it is complete, meaning that no observation were missed from any of the cases within the set. Table 1

provides the resulting instantiations of each of the input variables as well as the output PL, resulting in a total of 40000 data points within the set. Figure 1 shows the distribution of PL across all of the data points in the set.

Table 1: List of Nodes in BN Acoustic Model

Node	**Range**	**Instantiations**
Range (m): R	$100:100:1000$	10
Depth (m): Z	$50:50:500$	10
Sea Floor SF:	flat (1), uphill (2), downhill (3), up and down (4)	4
Bottom Density (g/cc): BD	$1.5:0.1:2.4$	10
Bottom Sound Speed (m/s): V	quadratic experiential function of BD	10
Source Frequency (Hz): F	$10:20:200$	10
Propagation Loss (dB): PL	discretized in model	10

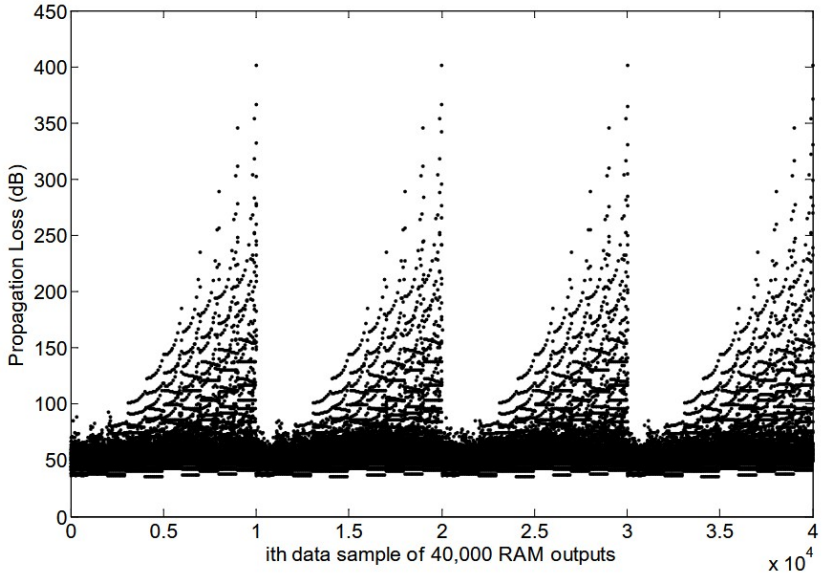


Figure 1: Resultant PL across the 40000 data points within the provided dataset

3 CONSTRUCTING THE NETWORK

To construct the BN and perform inference, the Bayes Net Toolbox (BNT) [3] was implemented in MATLAB.

3.1 PREPROCESSING THE DATA

To process the data and prepare it for the creation of the BN, first the data was uploaded into the program. Next, the PL data had to be discretized into separate "bins" so that the BNT could properly handle the points within its functions. While the toolbox can handle continuous variables, it fails when there are *both* continuous and discrete variables within the set. So, within the set, as the input instantiations were chosen regularly, they are known to be discrete variables, while the PL result from simulation is a continuous real number. To discretize the PL, 10 logarithmically-spaced bins were chosen such that any point within that bin was assigned a single discrete value, regardless of internal spread or variance. The reason that logarithmically-spaced bins were chosen over linearly-spaced bins is that the data in Figure 1 is *not* linearly distributed, with more points concentrated near the lower end than the higher end. Therefore, logarithmic spacing allows "wider" bins near the top to increase the number of points in them, hopefully giving more weight for structure/parameter learning for these sparse data regions. However, regardless of the bins, discretizing the PL data has the potential to greatly decreases the performance of the BN (as will be discussed later), as the feature size of the PL is massively diminished, but it is a limitation that must be implemented to properly interface with the MATLAB BNT.

Once all the variables were discretized, the data points were randomly shuffled and split into a training set and testing set at (80%/20%), respectively. Shuffling the data first ensures that the training should not be biased by variable instantiation order within the data set, providing a more generalizable result.

3.2 DETERMINING THE NETWORK STRUCTURE

To construct the BN, first the structure of the connection between the nodes had to be learned. To do this, the BNT's "greedy" K2-algorithm was implemented (`learn_struct_k2` [3]). In this algorithm, initially each node has no parents. Then, parents are added incrementally in a way that such that each addition of a parent is the one that most increases the score of the resulting structure (with the score of a particular BN graph being defined from a Bayesian scoring metric [3]). When no parents can increase the score, it moves to the next node and repeats. One caveat of using this method is that the nodes need to be given a prior topological ordering, where parents *must* come before child nodes in the ordering. For our cases, however, this is simple to achieve, as it is known that all the nodes have the potential to become parents for the PL node, so in the ordering, PL is last. In the case that the order is not known, a different search algorithm must be used. Figure 2 gives the final learned structure of the BN for 10 logarithmically-spaced PL bins. As can be seen, from the training data and K2 algorithm only **three** nodes were found to be parents to the PL loss (node 7). Those nodes are: frequency F (2), depth Z (5), and range R (6). During testing, this was the structure that was always found, regardless of if the discretization bins were set to be linear or logarithmic, and even if the number of bins were increased. The reason for this is most likely due to the poorly-shaped and distributed nature of the data, as through discretizing the PL, *most* of the resulting points will be in the first few bins, and so other such variables (i.e., sea floor SF, bottom density BD, and bottom sound speed V) are "absorbed" into the discretization, where their changes do not cause the data to fluctuate

greatly enough to "hop" bins, thus removing their perceived influence on the PL. This is one of the major downsides for discretizing a continuous variable for the sake of BN training. As previously mentioned, the "feature space" of PL is reduced to such a degree that certain variables no longer have an impact, limiting the usefulness of the network for inference to the "solitary" nodes.

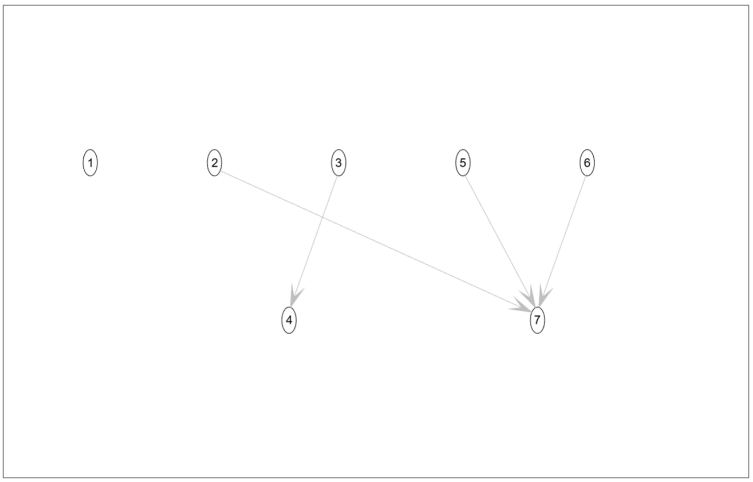


Figure 2: Resulting BN structure learned for 10 logarithmically-spaced PL bins

3.3 PARAMETER LEARNING

Once the network structure is learned from the data, the actual parameters (which are the CPTs corresponding to each node) must be learned from the data. To do this, the BNT's maximum likelihood algorithm ($\hat{\Theta} = \{\hat{\theta}_{ijk}\} = \arg\max_{\{\theta_{ijk}\}}\{p(T|S,\Theta)\}$) was implemented (`learn_params` [3]). In this algorithm, the network was initialized with random CPT parameters, which are then optimized over to find the parameter arguments that maximize the likelihood of recovering the training data T given the structure S and parameters Θ. In relation to the PL (node 7), the resulting CPT is a table that has $10 \times 10 \times 10 \times 10$ entries, which is far too large to include in this right up, but it contains the conditional probabilities of the PL being in "bin" $i \in \{1, 10\}$ given the discrete value of the range R, depth Z, and frequency F.

4 RESULTS AND ANALYSIS

Now that the parameters of the BN are learned, it is complete and can be used to perform inferencing.

4.1 INFERENCING

For this assignment, the inference that was performed was to predict the PL given the other six variables. Note, in the case of the network in Figure 2, only three of the variables have

any effect on the predicted output, but for completion's sake, all six other variables were instantiated to arrive at a prediction. The evidence that was input into the completed BN were the data points from the *testing* set. They were instantiated by, for each point, creating an inference engine (`jtree_inf_engine` [3]), including the `evidence`, then calculating the "most probably explanation" (`calc_mpe` [3]) given the evidence and engine. This function determines the most-probable discrete outcome of the PL given the observations, rather than providing a probability distribution for the final confidence of the PL in each state. This is the standard method for actually selected a determination from the prediction, as the resulting distribution is not important for the sake of this assignment. Additionally, as the data is *perfect* from simulation (and contains no stochastic noise), the predicted distributions, using the generated BN, were always *singular*, as in perfectly deterministic to a single output value for PL, so they provided no additional information, regardless.

4.2 SCORING METRICS

To determine the performance of the BN, four difference metrics were utilized to compare the actual testing PL ($\tilde{PL}$ from the predicted PL ($\bar{PL}$):

4.2.1 MATCH ACCURACY (MA)

The case match accuracy ($\hat{MA}$) of any given prediction $\bar{PL}$ compared to the respective true value $\tilde{PL}$ is given as

$$\hat{MA} = \begin{cases} 1 & \bar{PL} = \tilde{PL} \\ 0 & \bar{PL} \neq \tilde{PL} \end{cases} \tag{3}$$

Therefore, for our total inferred PL vector $\underline{\bar{PL}}$ and total testing PL vector $\underline{\tilde{PL}}$, the average match accuracy MA is given as

$$MA = \frac{1}{N}\sum_{i=1}^{N} \hat{MA}_i = \frac{1}{N}\sum_{i=1}^{N} \begin{cases} 1 & \bar{PL}_i = \tilde{PL}_i \\ 0 & \bar{PL}_i \neq \tilde{PL}_i \end{cases} \tag{4}$$

where N is the total number of testing data points. This method is very straightforward to compute to determine the overall accuracy of the system, however it has disadvantages that it does not consider the "error distance" between the prediction and the actual PL, and as a result it cannot be used to determine the confidence interval of the model.

4.2.2 DISTANCE METRIC (DM)

The case distance metric ($\hat{DM}$) of any given prediction $\bar{PL}$ compared to the respective true value $\tilde{PL}$ is given as

$$\hat{DM} = |\bar{PL} - \tilde{PL}| \tag{5}$$

Therefore, for our total inferred PL vector $\underline{\bar{PL}}$ and total testing PL vector $\underline{\tilde{PL}}$, the

average distance metric DM is given as

$$DM = \frac{1}{N}\sum_{i=1}^{N} \hat{DM}_i = \frac{1}{N}\sum_{i=1}^{N} |\bar{PL}_i - \tilde{PL}_i| \tag{6}$$

The advantage of this scoring metric is that the distance between the predicted PL and true PL is captured by the result, and when averaged it provides some measure of the "mean" distance between any one prediction and its respective true value. However, this method still does not consider confidence intervals.

One thing to note about computing DM is that the output/inference of the BN is a *discrete* integer giving the bin number that the PL is most likely to reside within. Therefore, to convert this integer value to decibels to match the PL given in (2), the inferred PL was given as the *central dB* value of its respective bin, allowing for a meaningful DM result.

4.2.3 DISTANCE EXPECTATION METRIC (DEM)

The case distance expectation metric ($\hat{DEM}$) of any given prediction $\bar{PL}$ compared to the respective true value $\tilde{PL}$ is given as

$$\hat{DEM} = |\bar{PL} - E[\tilde{PL}]| \tag{7}$$

where $E[\tilde{PL}]$ is the expected value of the true PL. We can find the expected value as

$$E[\tilde{PL}] = \sum_{i=1}^{B} p(\tilde{PL}_i = PL_B) PL_B \tag{8}$$

where B is the number of discrete bins and PL_B is the propagation loss within that bin. To find the actual probability distribution $p(\tilde{PL} = PL_B)$, the frequentest approach was used, where

$$p(\tilde{PL} = PL_B) = \frac{1}{N}\sum_{i=1}^{N} \begin{cases} 1 & \tilde{PL}_i = PL_B \\ 0 & \tilde{PL}_i \neq PL_B \end{cases} \tag{9}$$

Therefore, for our total inferred PL vector $\underline{\bar{PL}}$ and total testing PL vector $\underline{\tilde{PL}}$, the average distance expectation metric DEM is given as

$$DEM = \frac{1}{N}\sum_{i=1}^{N} \hat{DEM}_i = \frac{1}{N}\sum_{i=1}^{N} |\bar{PL}_i - E[\tilde{PL}_i]| \tag{10}$$

The advantage of this scoring metric over MA and DM is that it now allows for the interpretation of the confidence interval of the results due to the computation of the expected value of PL and the frequentest probability distribution for each bin. As with DM, because the BN only predicts which bin the PL will reside in, the discrete value is also given a corresponding dB value such that the result is physically meaningful.

4.2.4 NORMALIZED DISCRETE DISTANCE EXPECTATION METRIC

The case normalized discrete distance expectation metric ($N\hat{DDEM}$) of any given prediction $\bar{PL}$ compared to the respective true value $\tilde{PL}$ is given as

$$N\hat{DDEM} = |\mathbb{B}(\bar{PL}) - \mathbb{B}(\tilde{PL})| * ND \tag{11}$$

where $\mathbb{B}(\cdot)$ denotes the operation of determining the discrete bin that PL resides in and ND is the neighbor distance

$$ND = \frac{2(0.5B - 1)}{(B-1)(B-2)} = \frac{1}{B-1} \tag{12}$$

where B is the number of discrete bins.

Therefore, for our total inferred PL vector $\underline{\bar{PL}}$ and total testing PL vector $\underline{\tilde{PL}}$, the average normalized discrete distance expectation metric $NDDEM$ is given as

$$NDDEM = \frac{1}{N}\sum_{i=1}^{N} N\hat{DDEM}_i = \frac{1}{N}\sum_{i=1}^{N} |\mathbb{B}(\bar{PL}_i) - \mathbb{B}(\tilde{PL}_i)| * ND \tag{13}$$

An advantage of this scoring metric over the others is that it is developed specifically for discrete variables and that it can be naturally extended for larger and larger discrete sets with minimal computation. However, unlike the others, for continuous variables (such as PL) it requires careful discretization to determine appropriate bins for each of the respective data points.

4.3 RESULTS

Table 2 gives the results of BN's performance on the testing data set with respect to each of the four metrics in (4), (6), (10), and (13).

Table 2: List of Nodes in BN Acoustic Model

Metric	**Score**
MA	0.8671
$DM\ (dB)$	2.9924
$DEM\ (dB)$	13.1646
$NDDEM$	0.0168

As seen, according to MA, the Bn performed quite well. However, a large reason for this is *because* of the discretization and the ill-prepared dataset. As the majority of the data points were placed into very few bins (mostly bins 1 and 2, with $B = 10$), the network only needed to "learn" parameters such that for any given evidence, the PL is most likely to be predicted in one of these lower bins to cause high accuracy. Because of this, it is difficult to determine if the model is actually good at predicting proper PL or if the results are simply a function of how congested the data distribution is. This is not just an issue for the MA.

Notably, it plagues all of the performance metrics, and while DM and DEM may provide more physically meaningful results, if all points are estimated to have a PL near the lower spread of data, then they too will suffer from falsely-high performance.

Overall, despite the shortcomings of the provided dataset and limitations of the BNT [3], a proper BN was able to be configured, constructed, and training given data and used to infer PL from six other environmental observations, thus demonstrating the powerful impact that this model can have in complex perception challenges.

REFERENCES

[1] Chenghui Cai, Silvia Ferrari, and Ming Qian. "Bayesian Network Modeling of Acoustic Sensor Measurements". In: *SENSORS, 2007 IEEE*. 2007, pp. 345–348. DOI: 10.1109/ICSENS.2007.4388406.

[2] P. C. Etter. *Underwater Acoustic Modeling: Principles, Techniques and Applications, Second Edition*. Taylor & Francis, 1996.

[3] Kevin Murphy. *Bayes Net Toolbox for Matlab*. Sept. 2026. URL: https://github.com/bayesnet/bnt (visited on 06/28/2014).